

StudyHelper – Project Description

Alexandru Savin(354790)

Cristina Aurelia Matei (354776)

Damian Michal Choina (354789)

Eduard Fekete (355323)

Jakub Maciej Baczek (354814)

Karina Rubahova (354565)

Mara-loana Statie (354536)

Marta Zrno (355351)

Piotr Junosz (355502)

Tymoteusz Krzysztof Zydkiewicz (355413)

Supervisors:

Erland Ketil Larsen

Joseph Chukwudi Okika

Kasper Knop Rasmussen

Markéta Tranberg

15707 characters

Software Technology Engineering

4th semester

5.3.2026

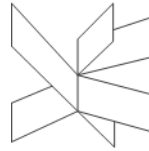
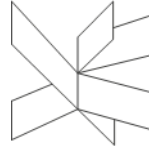


Table of content

1. Problem Domain	1
2. Problem statement	2
3. Delimitation	3
4. Choice of methods	4
5. Time schedule	5
6. Risk assessment	6
7. References	7



1. Problem Domain

The physical environment in which the educational process takes place has been the subject of study in recent years, given its direct impact on the well-being and academic performance of students (Bustamante-Mora et al., 2025, para. 2.).

Environmental conditions such as temperature, noise, lighting, and air quality can have either a positive or negative effect on the ability to study, even when students have the required motivation.

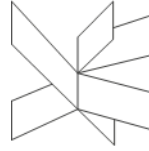
This can cause students to change places, adjust lighting, change clothing, open windows, or use other temporary solutions to make the environment more comfortable. These actions can waste time and energy and may interrupt the study process.

This problem is particularly visible in shared spaces, such as libraries, classrooms, and dormitories, where students do not always have control over the environment.

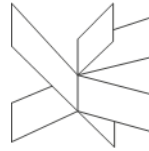
For example, many students report a lack of focus in a noisy space due to a lack of soundproofing. This forces them to rely on solutions such as noise-cancelling headphones or white-noise machines. Unfortunately, these solutions are not widely available, meaning that many continue to struggle with subpar conditions.

Students are not the only ones affected by this issue. Teachers also work in these environments and depend on productive learning conditions. The main stakeholders in this problem are students and teachers, who both need comfortable and productive learning environments.

The difficulty is that these conditions are often invisible or hard to judge accurately. A student may feel tired, distracted, or uncomfortable without knowing whether the cause is related to CO2 levels, temperature, lighting, or another environmental factor. Because of this, students and teachers often react only after the environment has already affected concentration or comfort.



There is a clear need for a system that connects raw environmental data with useful feedback about study-environment suitability. By integrating IoT sensors with machine learning, the system can move beyond simple data logging and provide decision support. The system can help users understand current environmental conditions, identify patterns over time, and support better decisions about where and when to study.



2. Problem statement

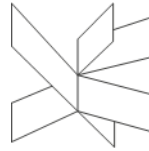
Based on the environmental challenges identified in the problem domain, there is a need for a technical solution that can track study conditions in real-time. This project addresses the following:

Main problem:

- How can indoor environmental data be monitored and analyzed to better understand how environmental conditions affect the quality of students' study sessions?

Sub-questions:

- What are the most significant indoor environmental factors that affect students' study conditions?
- What factors can be used to represent study session quality and indoor environmental quality?
- How do students react to changes in indoor environmental quality?
- Is there a measurable relationship between environmental factors and user-provided study ratings?
- How accurately can study suitability be predicted based on collected sensor data?



3. Delimitation

The system is designed around a physical IoT device that can be connected to a user account and used in different study environments, such as classrooms, libraries, or personal study spaces. The following delimitations were agreed at project inception and are reflected in the implemented system:

1. Geographical Scope

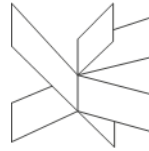
- The device is designed and tested within a campus-related study context, such as classrooms, libraries, or personal study spaces. The IoT device is treated as a portable physical sensor that can be connected to a user account and used in the environment where the user is currently studying. Multi-building deployment is out of scope for this iteration.

2. System Features

- **Sensor coverage:** The system measures temperature, humidity, CO₂ concentration, and light. Sound level measurement is not currently implemented by the active firmware, despite being noted as a candidate sensor in the project description due to the malfunctioning and inaccurate quality of the sound sensor.
- The system does not attempt to measure or infer physiological state, stress levels, or cognitive load directly. The Study Suitability Rating is an environmental proxy, not a clinical assessment.
- **Automation:** The system provides predictive guidance but does not automate any physical actuators beyond the onboard buzzer alert triggered when the predicted study quality rating reaches the lowest level. Full HVAC or lighting automation is explicitly out of scope.

3. Actors and Stakeholders

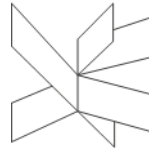
- **User model:** The system supports user roles such as student who can start and stop sessions via the physical button on the



device and submit a post-session quality rating through the frontend, and a teacher who assesses whether conditions are suitable for teaching. Administrator views are out of scope.

4. Data and Privacy

- The system stores environmental sensor readings and session lifecycle metadata. In addition, user accounts are stored, including name, last name, email address, and a hashed password, as well as optional profile fields such as university, study programme, study year, study goal, and a profile picture reference.
- Connected device information and post-session ratings submitted by users are persisted so ratings can be associated with the correct device and study session.
- No audio recordings or video are collected.
- All personally identifiable data is linked to a user account and is subject to standard data protection considerations; password storage uses hashing.



4. Choice of methods

4.1 Development Methodology

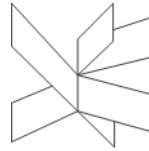
The development of the system will follow the Agile Unified Process (AUP) combined with Scrum as the project management framework (Ambler, 2006). Thanks to AUP, the development process will be structured into the phases Inception, Elaboration, Construction, and Transition. This process supports iterative and incremental development, allowing the team to improve the system architecture and functionality step by step during the project (Ambler, 2006).

For development coordination, Large-Scale Scrum will be applied (Larman & Vodde, 2016). The team is split into three sub-teams: IoT, Machine Learning, and Front-End, all working within the same sprint cycle. Each team has its own Scrum Master, while a single Product Owner manages the overall product backlog and prioritizes system requirements (Larman & Vodde, 2016).

Sprint durations will depend on the development phase. During the Elaboration phase, the teams will work in two-week sprints, while during the Construction phase the sprint duration will be one week to support faster implementation and integration. Each Scrum team will conduct its own ceremonies and adjust them according to the team's needs.

4.2 Requirements Analysis

The team defines system requirements through user stories and use cases. The team writes user stories according to the SMART principles (Doran, 1981). Each story defines a clear goal and measurable outcome. This structure guides development and supports clear task definition.



Use cases will be used to describe how different users interact with the system. For example, a student should be able to view environmental data, start or follow a study session, and receive feedback about study conditions. These use cases will help the team understand the main system behavior from the user's perspective.

The team creates several UML diagrams to support system analysis and design. These include Use Case diagrams, Sequence diagrams, Activity diagrams, a Domain Model, and Class diagrams. These diagrams describe system behaviour and structure. This should make it easier for the different subteams to understand how their parts connect.

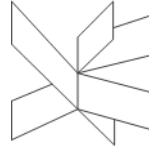
Because the project includes frontend, backend, IoT, and machine learning components, requirements may need to be adjusted during development. If technical limitations or integration issues appear, the team will update the requirements and diagrams so they stay aligned with the actual solution.

4.3 Development and DevOps Practices

The project follows DevOps practices to support collaboration and continuous integration between teams (Bass, Weber & Zhu, 2015). The team stores the source code in Git and uses GitHub as the central repository. Feature branches and pull requests will be used so that changes can be reviewed before they are merged into shared branches.

Automated tests run during development. These tests check whether new code changes break parts of the system which already worked correctly. This is important because the project consists of several connected components, and changes in one part may affect another part. Continuous integration should help the team detect these problems earlier.

Docker containers will run parts of the system where needed. Containers keep the development and runtime environments consistent, especially for the backend,



database, frontend, and machine learning service. This should make it easier for different team members to run the system in a similar way.

The backend and other system services will be deployed on a public cloud provider. This will make it possible for the frontend, IoT device, backend, and machine learning service to communicate outside one local computer. A cloud deployment will also make integration testing easier, because all teams can work against the same hosted environment.

The team also plans to use GitHub Actions or similar automation for building, testing, and deployment where possible. This should reduce manual work and make the project more reliable as it grows.

4.4 System Implementation

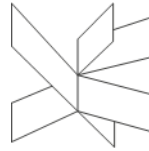
The system includes several main components. The IoT hardware platform uses the ATmega2560 MCU. The team programs the device in C. The device collects environmental data from sensors and sends the data to the backend system.

The backend is developed in C#. The backend exposes functionality through a REST API and handles communication between the IoT devices, the database, and the web application.

The system stores data in a PostgreSQL database. The database stores sensor measurements and other system data.

The machine learning component processes the collected sensor data and generates predictions based on the observed patterns. The team develops the model in Python.

The frontend is a responsive web application developed in React. The application displays sensor data, predictions, and historical information to the user.



4.5 Testing

Testing takes place during the entire development process. The team writes unit tests for individual components of the system (Beck, 2003). These tests check whether each component works as expected.

Automated tests also run during development. These tests check whether new code changes break existing functionality before the code is merged. This is important because the system will include several connected parts, such as the frontend, backend, IoT firmware, database, and machine learning service.

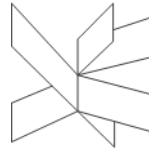
Different parts of the system will require different testing approaches. The frontend should be tested through component and interaction tests, for example checking page rendering, forms, navigation, language/theme switching, and important user flows. The backend should be tested through API tests that verify endpoints and data validation. The IoT part should include tests for hardware-independent logic where possible, while hardware-related behavior will also need manual testing on the physical device. The machine learning part should be tested by checking data processing, model loading, and prediction responses.

The goal of the testing strategy is to detect errors early, make development safer, and give the team more confidence when changing or extending the system.

4.6 Project Management and Communication

Project tasks will be managed using GitHub and Figma. These tools support task tracking, collaboration between teams and design of the user interface.

GitHub will be used for version control, branches, pull requests, code reviews, and tracking changes in the project. Figma will be used mainly for UI ideas, design discussions, and visual planning.



Communication within the development team will primarily take place through Discord, which will be used for daily coordination, discussions and information sharing between the teams.

The team will work with feature branches so that different parts of the system can be developed in parallel. Changes will be merged through pull requests, allowing other team members to review the work before it becomes part of the shared codebase. This should reduce the risk of conflicts and make it easier to understand what has changed.

The team also plans to use meetings and sprint planning to divide work, discuss progress, and identify blockers. This will help the group stay aligned, especially when one team depends on functionality from another team.

5. Time schedule

Number of group members	13 members
ECTS per student	10 ECTS
Expected workload per ECTS per student	28 hours
Sprint number of days between 16th of March and 4th of May	14 days
Sprint number of days between 4th of May and 28th of May	7 days

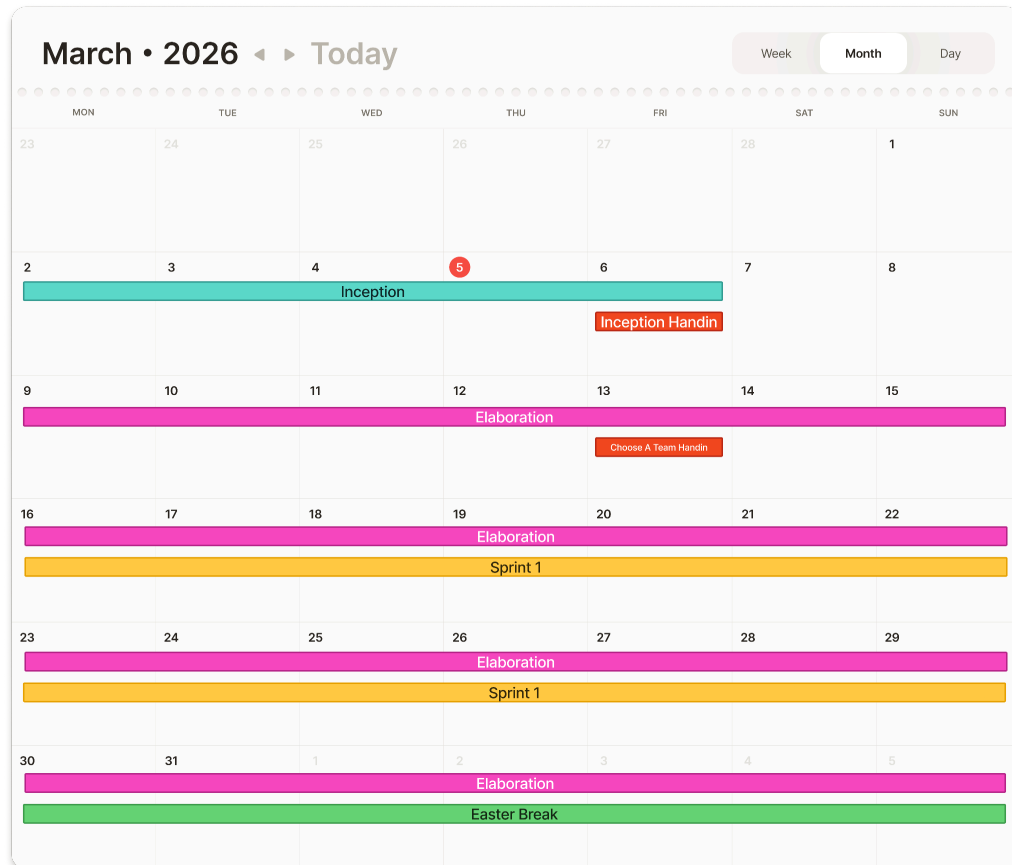
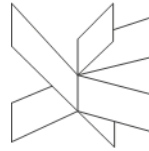


Figure 1 - March Schedule

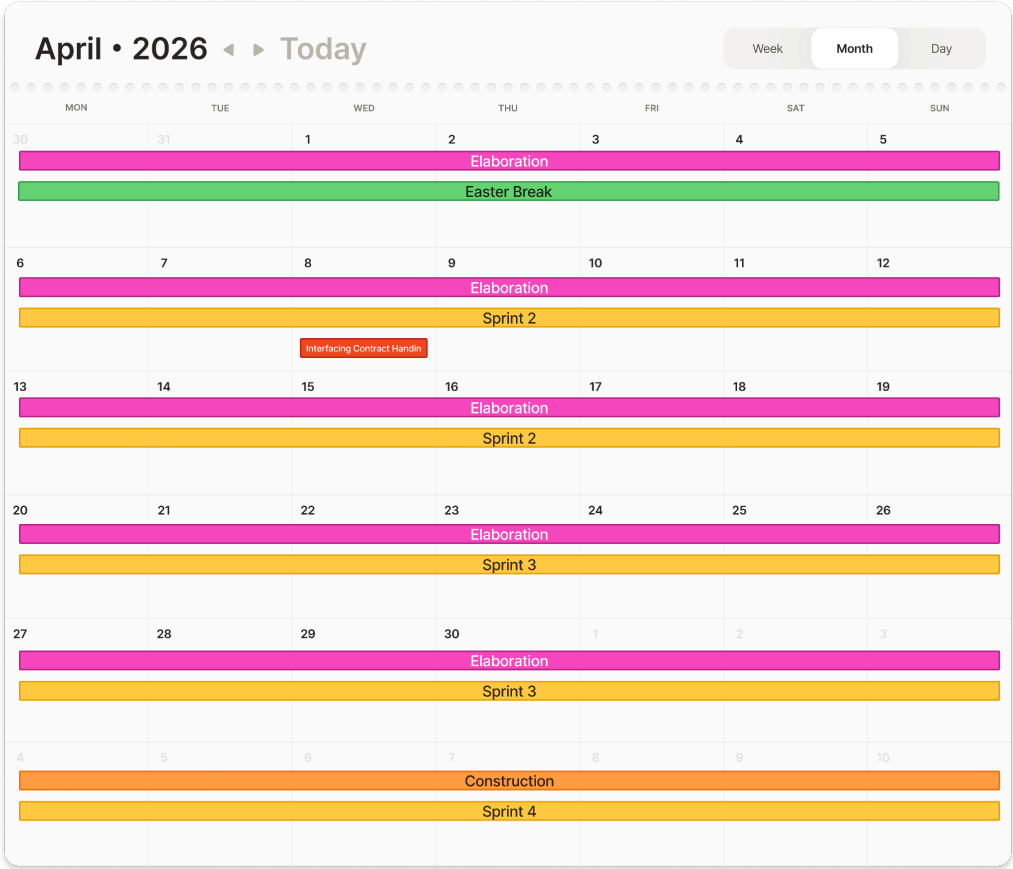
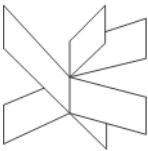


Figure 2 - April Schedule

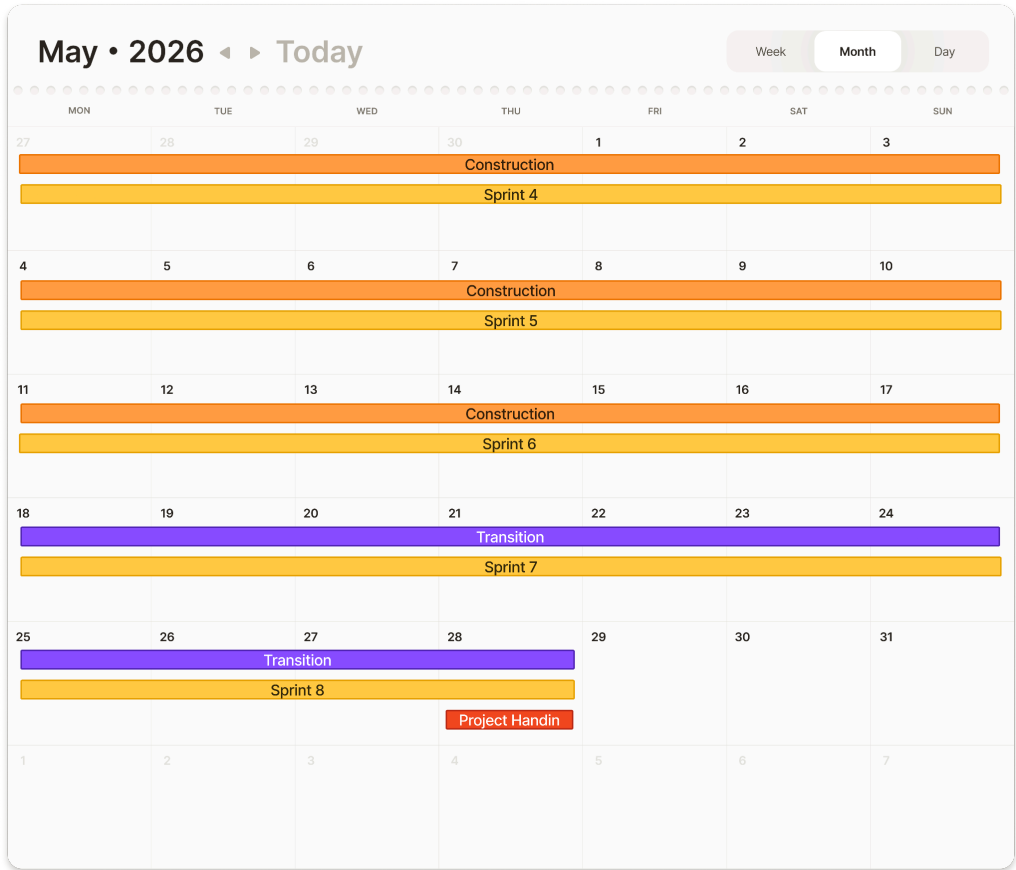
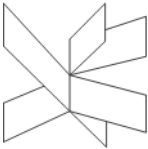
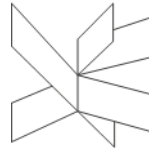


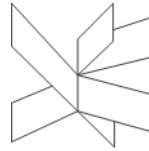
Figure 3 - May Schedule

Figures 1–3 show the detailed schedules for all planned Agile Unified Process development stages (Larman & Vodde, 2016). The figures also include the planned sprint timelines and the corresponding hand-in dates.



6. Risk assessment

Risk description	Likelihood (0 - 5)	Impact (0 - 5)	Risk Score (Likelihood x Impact)	Preventive Actions	Identifiers	Responsible
Insufficient dataset for ML model training	4	5	20	Create dummy data	Low model accuracy	Eduard Fekete
Late integration may occur if each group prioritizes its internal objectives without regularly aligning with other teams	4	5	20	Hold regular meetings to review and define clear integration deadlines	Components work separately but fail when combined	Eduard Fekete
A delay in one team may block other teams from making progress	4	4	16	Set shared milestones and monitor progress regularly to identify and address delays early	Teams waiting for deliverables from another team	Eduard Fekete
Hardware malfunction in IoT team	3	4	12	Perform hardware test as early as possible	Unstable or unrealistic sensor readings	Jakub Baczek
Decision-making delays due to large team size	2	3	6	Use majority voting when consensus cannot be reached within a defined discussion period	Long discussion without clear decisions	Jakub Baczek



7. References

- Bustamante-Mora, A., Diéguez-Rebolledo, M., Zegarra, M., Escobar, F., & Epuyao, G. (2025). *Environmental conditions and their impact on student concentration and learning in university environments: A case study of education for sustainability*. **Sustainability**, 17(3), 1071. <https://doi.org/10.3390/su17031071>
- Ambler, S. W. (2006). *The Agile Unified Process (AUP)*.
- Beck, K. (2003). *Test Driven Development: By Example*. Addison-Wesley.
- Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley.
- Doran, G. T. (1981). There's a S.M.A.R.T. way to write management's goals and objectives. *Management Review*.
- Larman, C., & Vodde, B. (2016). *Large-Scale Scrum: More with LeSS*. Addison-Wesley.